Shorter proofs for K215 and a checked transfer to CM215

Research packet for the ProveIt repository

21 October 2026

Abstract

The best immediate simplification is to make small machine transformations carry small invariant proofs, while retaining the existing logical calibration. We implement this for the six-row conditional-marker variant CM215. Lean 4.32.0 checks an arbitrary-tape simulation and, through a new exact semantics bridge, the theorem that the literal dense CM215 machine halts on blank input if and only if the repository's literal dense K215 machine does. A separate two-line adapter composes this with the existing K215/ZFC theorem; the full inherited ZFC dependency closure is not rebuilt in this packet.

We also supply checked proof-data reductions: sharing repeated Lean source subtrees reduces one generated file by 46.3%, and a structural binary path codec reduces a Rocq certificate file by 88.1%. Coq checks exact equality with the old certificate. A new symmetry theorem for the actual Rocq offset checker eliminates the second certificate computation. A newly checked ordinary source bisimulation removes the integer-offset checker from the K215 source lane and reduces its delivered data representation by 50.9%. New bounded searches of CM215, the final rebalanced candidate, and the final direct-transfer candidate do not yield a 214-state machine. We identify a second marker-fold opportunity and the precise continuation-provider problem preventing the simplest implementation.

1 Results and exact scope

This packet examines ProveIt at commit

`b6c7087477e51c017003bd729e712c8969733223`.

All statements about the repository below refer to this immutable snapshot, not to an unspecified future `main`. The user's goal is a deterministic, one-tape, two-symbol Turing machine whose blank nonhalting is provably equivalent in ZFC to syntactic consistency of ZFC. The state convention counts every working state, including initialization, and excludes the separate halt. The head moves only left or right on a two-way infinite tape, initially entirely zero.

Question	Result of this investigation
Shorten the existing proofs?	Yes. Exact shared-tree and path-codec prototypes, a checked Rocq symmetry theorem, and a source-checker specialization are supplied. Existing simplifications at the inspected commit are distinguished from these contributions.
Another 215-state candidate with an easier proof?	Yes: the already published CM215. We now provide a newly compiled exact dense-machine transfer theorem, based on a small local invariant, instead of repeating its full primitive-runtime proof.
A 214-state construction?	No. The new searches and a precise near miss are retained, but no table is represented as a verified 214-state result.
New complete ZFC formalization?	No. The existing K215/ZFC theorem is reused. The newly compiled theorem reaches the exact existing K215 machine; the final adapter importing the full ZFC development is supplied but not compiled here.

The distinction in the last row is deliberate. A separately compiled proof of an exact machine equivalence, together with an existing proof of the endpoint, is enough for a short mathematical composition. It is not the same build record as compiling a fresh final module with the whole inherited dependency closure. The packet does not claim an internal PA derivation for K215 or CM215, unconditional nonhalting, or a lower bound of 215 states.

2 The present proof is already substantially factored

The current Lean endpoint is `KEndpoint215.blank_halts_iff_zfc_refutation`:

$$\texttt{BlankHalts machine215} \quad \longleftrightarrow \quad \texttt{BProv ZFCax_s [] Form.fBot}.$$

The Rocq endpoint is `K215F.CorrectDense215.tm215_halts_iff_ZFC_inconsistent`, with the corresponding explicit classical derivability predicate. The nonhalting theorems are the negations of these equivalences. The exact source paths and retained verification records appear in the references and in `proof_audit/audit.md`.

Here "ZFC endpoint" names the external proof-assistant theorem relating machine halting to the explicitly defined ZFC refutability predicate. This packet does not emit an object-language ZFC or PA derivation code. The new marker transfer uses finite transition-table facts and induction on finite runs, so its ordinary mathematical proof introduces no additional set-theoretic strength. Once the original calibration is available in ZFC, that transfer composes with it there. The freshly compiled artifacts establish the exact machine-equivalence part in Lean; the separate internal-derivation question is not silently counted as checked.

The original verification record reports a fresh 204-module Lean project closure, two endpoint clients, and a 237-source Rocq build followed by recursive `coqchk`. The latest simplification checkpoint reports a 185-module primary Lean closure: it recompiles affected modules and explicitly binds the reused objects to their sources. These are different records. We have inspected them, not rerun either entire endpoint closure.

The repository already implements the following reductions:

Component	Earlier bytes	Current bytes
Lean dominated implication	689,857	35,185
Lean AX routine	202,222	7,563
Lean source data	653,569	456,548
Lean site-coverage facade	107,243	14,916
Lean register determinism	17,209	12,988
Rocq dominated implication	921,691	22,673

These are the repository authors' existing results. In particular, Lean already obtains the reverse offset simulation from symmetry and has removed sixteen reverse-certificate modules. The AX proof already composes a word of macro contracts; the enumerator proof already shares a general theorem between starts. Merely recommending these changes again would not advance the current state.

The final machine-specific Lean endpoint file has only 57 lines and calls a generic endpoint theorem. Its arguments already separate the dense renaming, register engine, blank bootstrap, source-site coverage, and program-level consistency calibration. The remaining opportunities are primarily in the generated representations, certificate checking, and proof transfer between nearby tables.

3 A general positive-step transfer principle

The reusable abstraction needed for nearby machines is weaker than step-for-step bisimulation and stronger than a finite test. It allows an old step to become a positive finite block of new steps, relates complete configurations at block boundaries, and ensures that synchronization times are cofinal.

Let $f : A \to A$ and $g : B \to B$ be total deterministic transition functions. A halted configuration may be totalized by making it a fixed point. Write $f^n(a)$ for iteration. Let P_A and P_B be the respective halting predicates, and $R \subseteq A \times B$ a relation.

Theorem 3.1 (Positive simulation transfers eventual halting)**.** *Suppose:*

1. *If $R(a, b)$, then for some integer $j \geq 0$,*

$$R(f(a), g^{j+1}(b)).$$

2. *If $R(a, b)$, then $P_A(a) \leftrightarrow P_B(b)$.*

3. *$P_B(b) \Rightarrow P_B(g(b))$.*

Then every related pair satisfies

$$(\exists n\ P_A(f^n(a))) \quad \longleftrightarrow \quad (\exists n\ P_B(g^n(b))).$$

No uniform upper bound on the block lengths is required.

Proof. Induction on n constructs a synchronization time $k_n \geq n$ with $R(f^n(a), g^{k_n}(b))$. The base time is zero. At a successor, append the positive block supplied by the first assumption, so $k_{n+1} \geq k_n + 1$.

If the source halts at time n, halting agreement gives target halting at k_n. Conversely, if the target halts at time n, choose the synchronization associated with source time n. Its target time k_n is at least n, so target halting persists to k_n. Halting agreement then yields source halting at time n. This is an existence argument about a halting source configuration, not an equality of source and target halting times. □

The strict positivity is essential: if empty blocks were allowed, infinitely many source steps could be related to one fixed target time. The supplied `PositiveSimulation.lean` encodes the length as `j + 1`; this prevents accidentally omitting the progress obligation. It contains the cofinal-synchronization and halting-equivalence theorems. Its proof does not use classical choice to select a global sequence of block lengths.

For invariant-based optimization, put the invariant inside R rather than maintaining a separate trace-history predicate. This makes the interface convenient for local changes:

$$R(c,d) \;:\Longleftrightarrow\; I(c) \;\wedge\; d = \Phi(c).$$

One then proves invariant preservation, the block simulation, and halting agreement. The generic theorem handles infinite runs and the reverse implication once.

4 Why CM215 is the best immediate alternative

CM215 is an existing repository candidate, not a newly invented machine in this packet. It is particularly attractive because its numerical calculus, source program, and startup obligations can be inherited through an exact machine equivalence. Other alternatives change the source layout, command semantics, startup frame, spare representation, or logical axiom emitter; those changes may ultimately be valuable, but their proof deltas cross more interfaces.

CM215 changes exactly these six *native-labelled* rows:

State	Read	K215	CM215
52	0	$0\,R\,154$	$0\,L\,176$
156	1	$1\,R\,176$	$1\,R\,52$
165	0	$0\,R\,176$	$0\,R\,52$
176	0	$0\,R\,168$	$0\,R\,185$
176	1	$1\,R\,150$	$1\,R\,129$
193	0	$0\,R\,176$	$0\,R\,52$

All remaining rows agree literally. The old roles 52 and 176 share their read-one action but differ on zero. CM215 merges their entry role into 52 and reuses physical label 176 to inspect a predecessor marker. This is state-neutral: the result still has 215 working states.

4.1 A three-clause inductive invariant

Represent a complete tape by $T : \mathbb{Z} \to \{0,1\}$ and a configuration by (q,h,T). For the *old* machine define

$$\begin{aligned} I(q,h,T) \;:\Longleftrightarrow\; & (q = 52 \Rightarrow T(h-1) = 1) \\ & \wedge\, (q = 176 \wedge T(h) = 0 \Rightarrow T(h-1) = 0) \\ & \wedge\, (q = 156 \Rightarrow T(h+1) = 1). \end{aligned}$$

The third clause is the useful strengthening. Without it, the exceptional arrival at 176 looks as though it requires inspecting two prior steps of the run. With it, preservation is a one-step local induction.

Lemma 4.1. *I holds at every configuration of K215 reached from control 0, for every initial tape and every integer initial head position.*

Proof. It is vacuous at control 0. Every arrival at 52 writes one and moves right, proving the first clause at the destination. An arrival at 176 either writes zero and moves right, which establishes its predecessor-zero clause, or comes from 156 and writes one while moving right. In the exceptional case, the third clause says the newly scanned cell is one, so the premise $T(h) = 0$ of the second clause is false. Finally, every arrival at 156 writes one and moves left; the freshly written cell is therefore the cell to the right of the new head. This proves the third clause. □

The literal incoming-edge facts are discharged by finite kernel decisions over both symbols and all native labels. The argument quantifies the entire infinite tape; no assumption about unary registers, spare blocks, formula encodings, or blank exterior is used.

4.2 The exact local simulation

Define the control map

$$\phi(q) = \begin{cases} 52, & q = 176, \\ q, & q \neq 176, \end{cases} \qquad \Phi(q, h, T) = (\phi(q), h, T).$$

On an old read-one transition from 52 or 176, both machines perform the same action to 150. On a read-zero transition, the new machine takes a three-step block.

At old 52, the invariant supplies predecessor one:

$$(52, h, T) \longmapsto (176, h-1, T) \longmapsto (129, h, T) \longmapsto (154, h+1, T).$$

At old 176 with current zero, the invariant supplies predecessor zero:

$$(52, h, T) \longmapsto (176, h-1, T) \longmapsto (185, h, T) \longmapsto (168, h+1, T).$$

Each transition writes back the symbol that it reads. Hence the whole tape is unchanged, not just a selected window. The final head and mapped control agree exactly with the old step. All other old rows are reproduced by one new step with their successor relabelled by ϕ.

Halting agrees at a synchronization boundary. States 52 and 176 are nonhalting in both relevant roles. Every other row is mapped consistently, including the absence of a row at the terminal. Halting is stable under the totalized transition function. The positive-step theorem therefore applies.

Theorem 4.2 (Checked exact-table transfer)**.** *For every tape $T : \mathbb{Z} \to \{0, 1\}$ and every initial head position h,*

$$\mathrm{Halt}(\mathsf{K215}, (0, h, T)) \quad \longleftrightarrow \quad \mathrm{Halt}(\mathsf{CM215}, (0, h, T)).$$

In particular, K215 and CM215 halt alike on the blank tape.

The new theorem is `CM215Exact.all_tapes_equihalt`. The proof applies only from the designated control 0, or more generally from configurations satisfying the displayed invariant; it does not assert equivalence from arbitrary intermediate controls. The block proof also shows that t old transitions correspond to $t + 2z$ new transitions, where z counts old read-zero visits to 52 or 176. This exact clock identity is an ordinary consequence of the local blocks; the delivered general theorem requires only positive durations.

4.3 State counts, sparse holes, and literal data

The native labels include 301 and 302, so the standalone formal model uses the ambient type `Fin 303`. This does *not* mean that the machine has 303 working states. The finite kernel checks prove that exactly 215 labels have outgoing rows, that both symbol rows exist at every working label, and that every successor is working or is the separate halt 254. Initial label 0 is working. Thus the unused labels cannot occur on a run from the designated start.

The dense machines use precisely 215 working states. Independent parsing compared both generated native Lean tables and the generated CM215 dense Lean table with the pinned text tables, and checked both complete state bijections. The compiled `old_compatible` theorem separately binds the actual imported `KDense215.machine215` to the pinned native K215 table. The exact input hashes are:

K215 native
`9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2`

K215 dense
`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`

CM215 native
`e5907f08e0752eada0e25bbf33680341e80e5fe94b52b409aea1ed039edda252`

CM215 dense
`1f4f084f7293e2b042d5746039f793ff72ed429083b504970c939a5515e666cf`

5 Connecting to the actual existing endpoint

A standalone theorem in another tape model is not enough by itself. We therefore prove a synchronous bridge from the repository's `BusyBeaver.Core` machine semantics to the function-tape semantics used above. It reads every cell of Core's finite-support tape, preserves the head position, and sends dense working states through the exact native-label map. The dense terminal is sent to native halt 254.

The bridge has three principal obligations:

1. Every dense row is exactly the corresponding native row after relabelling its successor.
2. Reading after a Core tape write agrees pointwise with the function-tape write.
3. The embedded dense configuration is halted if and only if the native configuration has no row.

These establish synchronous iteration by induction. Finite compatibility with the old and new literal tables is checked by ordinary Lean `decide`; the old dense object is the actual imported `KDense215.machine215`, not a separately asserted equivalent copy.

The newly compiled result is

```
CM215Core.blank_halts_iff_primary :
  KDenseBridge.BlankHalts CM215Core.machineCM215 <->
  KDenseBridge.BlankHalts KDense215.machine215
```

It uses only `propext` and `Quot.sound`, according to the emitted axiom report. The exact dense-row compatibility proofs use only `propext`. The small build uses Lean 4.32.0 and no mathlib dependency. It includes the six unchanged upstream Core/zipper/table/dense-bridge modules necessary to bind to the existing machine.

The final repository adapter is then simply

```
theorem blank_halts_iff_zfc_refutation :
    BlankHalts CM215Core.machineCM215 <->
      BProv ZFCax_s [] Form.fBot :=
  CM215Core.blank_halts_iff_primary.trans
    KEndpoint215.blank_halts_iff_zfc_refutation
```

Negation gives the nonhalting/consistency theorem. This adapter is supplied in `formal_cm215/CM215ZFCAdapter.lean`. Its full import closure was not rebuilt here, so its status is an integration artifact, not a fresh end-to-end endpoint build. The underlying dense-machine transfer was compiled, its exact tables were independently audited, and the inherited endpoint is identified by immutable source pin.

This is the recommended next candidate to formalize. It needs no new proof of dominated-calculus completeness, source enumeration, protected stack credit, or startup calibration. Those components are reused through the exact machine theorem. The new reusable mathematical proofs occupy about four hundred lines across the positive-simulation, marker, and Core-bridge modules; the longer literal table files are data rather than repeated semantic arguments.

6 Exact compression of proof representations

6.1 Share source subtrees before elaboration

The current `KSource215Data.lean` still writes two fully expanded tree literals. Each has 4,751 primitive leaf occurrences. The optimized source tree has height 47 and 202 distinct subtrees; the structured reference has height 178 and 283 distinct subtrees. Their union contains only 427 distinct subtrees, so 58 are common to both.

The supplied generator interns the union and emits 427 acyclic private definitions in dependency order. Each public tree becomes its root constant. This changes how the term is written, while preserving the complete expanded abstract syntax tree. It leaves the rest of the original production file byte-for-byte unchanged, including the certificate and theorem interfaces.

The full generated file shrinks from 456,548 to 245,190 bytes, saving 211,358 bytes, or 46.30%. A separate parser recursively unfolds every emitted alias and compares the resulting syntax trees exactly. The standalone Lean file `SharedTreeIdentity.lean` additionally proves both original-expanded versus shared-root equalities by `rfl` in a local representation type; the ordinary source packet below separately checks the actual repository program bindings. Both identities were compiled with no axioms.

This is a measured source-file reduction. It is not a claimed 46% reduction in kernel checking time, proof-object size, or the total proof development. The full production module still needs its normal downstream rebuild before adoption. Sharing is attractive because its correctness can be checked at a small representation boundary and because it reduces repeated syntax independently of any search heuristic.

6.2 Encode paths structurally, with an explicit sentinel

The original Rocq `Cert215Offsets.v` is 1,499,170 bytes. It contains 2,240 paired checkpoints and 4,480 repeatedly written Boolean paths. Every stored offset in this particular certificate is zero. The longest path has 176 bits.

Use the positive binary inductive type as an exact path representation:

$$\begin{aligned} \mathrm{pack}([]) &= xH, \\ \mathrm{pack}(0 :: P) &= xO(\mathrm{pack}(P)), \\ \mathrm{pack}(1 :: P) &= xI(\mathrm{pack}(P)). \end{aligned}$$

The inverse recursively removes the outer constructor. The xH sentinel preserves every zero bit, including zeros at either end of a path; there is no fixed depth limit. Induction gives

$$\mathrm{unpack}(\mathrm{pack}(P)) = P.$$

The printed positive numerals are merely a compact notation for these structural constructors. The bit order is explicitly least-significant-bit first; it is not silently identified with some other source address convention.

The compact representation stores the two path codes and the two successor indices. The decoder restores the zero offset, so the final value has exactly the old checker type. The generated file is 179,009 bytes: a saving of 1,320,161 bytes, or 88.06%. Coq 8.18.0 compiled the original and compact modules and proved

```
CompactCertificate.Cert215 = OriginalCertificate.Cert215
```

by `reflexivity`. Both the codec theorem and this entire certificate identity are closed under the global context. Recursive `coqchk` also passed. This makes the generator untrusted: the correctness claim is about equality of the actual generated terms.

The equality transports any proposition about the old certificate to the new one without requiring a second semantic soundness theorem. Production still needs a build with the repository's pinned Rocq 9.1.0 toolchain and its normal source/hash records. The local Coq check is reported at its actual version, not as a replay of the original 237-module Rocq endpoint.

7 Remove the second Rocq certificate computation

The existing Rocq `Source215.v` computes both `certificate_forward` and `certificate_backward`, using a mirrored certificate for the latter. The inspected Lean proof already avoids this duplication; the Rocq lane does not.

For a checkpoint (P_1, P_2, z, j, k), define the mirror as

$$(P_2, P_1, -z, j, k).$$

The successor indices are unchanged: both programs test the same register, so the nonzero and zero outcomes retain their roles. The arithmetic balance condition for a segment is

$$\delta_z(i) + c_1(i) = c_2(i) + \delta_{z'}(i),$$

where only register 5 may carry the offset. Negating and rearranging gives

$$\delta_{-z}(i) + c_2(i) = c_1(i) + \delta_{-z'}(i).$$

The tested-register condition also survives, since $\delta_z(r) = 0$ implies $\delta_{-z}(r) = 0$. Lookup into the mirrored certificate is just lookup followed by swapping the paths and negating the offset.

The new `OffsetMirror.v` proves, for the actual imported checker definitions,

```
ocheck t1 t2 C = true ->
  ocheck t2 t1 (omirror C) = true
```

and hence the new interface

```
ocheck_iff_of_forward p1 p2 C :
  ocheck (ptree p1) (ptree p2) C = true ->
  forall R, rm_halts p1 ([],R) <-> rm_halts p2 ([],R).
```

Both theorems compiled and passed recursive `coqchk`, with closed assumption reports. The proof uses the exact pinned upstream `RM.v` and `Bisim230.v`, and the exact current K215 `OffsetBisim230.v`. It changes no existing soundness theorem.

Adopting this interface replaces the second `vm_compute` proof with an application of a general theorem. It removes duplicate checking work; no numerical build-time speedup is claimed without a production measurement. The compact path representation and symmetry theorem can be adopted separately, making regressions easier to isolate.

8 Use the weakest adequate source checker

Every one of K215's 2,240 source-relation offsets is zero. The natural next simplification is to use ordinary segment bisimulation instead of its integer-offset extension for this one source comparison. Its relation equates the complete natural register files, and each paired segment need only increment each register equally often. The tested-register offset side condition disappears.

There is a concrete trap. The existing ordinary Lean checker decodes paths with fixed fuel 128, while 622 of the 4,480 path occurrences in this certificate exceed 128 bits. Deleting the offset field and directly reusing that decoder would not faithfully decode this literal certificate and cannot be accepted without replay. This concerns the intended data binding, not the soundness of the ordinary checker: a successful check still certifies the sites and branches actually decoded. The longest source-reference path is 176 bits. A structural decoder, or a decoder whose fuel is demonstrably sufficient for its input, must be introduced at the checker interface first.

A robust interface parameterizes the ordinary checker by a function from path codes to actual Boolean paths. Its soundness theorem concerns the paths returned by that function, so it needs no bounded-codec completeness assumption. For intended literal bindings, the chosen decoder is separately checked against the actual source paths. This cleanly separates semantic soundness from representation fidelity.

The relation must also remain a relation, rather than being silently replaced by a bijection of control-flow vertices. In the current certificate there are 2,154 distinct left paths and 2,232 distinct right paths; 86 left paths and eight right paths occur twice. A canonical control-flow representation would therefore need a bisimulation quotient, not a naive graph-isomorphism check. Equality of the flattened instruction words is insufficient because break/continue targets depend on the tree context.

8.1 A newly checked ordinary source theorem

The delivered `plain_source/` implementation has now passed its complete isolated source-level replay with Lean 4.32.0. The final theorem is

```
KSource215PlainAudit.source_halts_iff_iter
    (R : RegisterPath.Registers) :
  RegisterPath.Halts KSource215.p215 ⟨[], R⟩ ↔
    RMDag.IterHalts DagPerm.dagPerm R
```

The theorem concerns every initial register file, not just the particular file used by the blank bootstrap. It first proves the exact public `KSource215.p215_halts_iff` interface between the actual source program and the actual path program `pathDagPerm`, then composes the unchanged path-to-iterative semantics theorem. The exact 487-node DAG well-formedness check is included.

The new generic checker, `RMPlainCodec.check_halts_iff`, reuses the ordinary bisimulation theorem. That theorem, `RMBisim.Bisim.halts_iff`, already includes symmetry. Its relation contains no integer offset. The selected import closure contains no `RMOffsetBisim`. Eight underlying semantics

and DAG modules are unchanged, hash-pinned copies of the repository modules. Both compact trees are bound to the actual source-program definitions by compiled exact equalities.

The chosen decoder is `totalDecode n := decodeAux n n []`. It uses input-dependent fuel rather than a fixed path-length cap; the actual recursion stops after the binary path has been consumed. All 2,240 entries, the root-entry obligation, and every branch were checked by the ordinary kernel. Independent parsing also confirms that each entry is exactly the projection $(a, b, 0, j, k) \mapsto (a, b, j, k)$ and that the original relation-tree structure and successor indices are retained.

The new generic checker has 113 lines and 4,639 bytes, compared with 293 lines and 13,151 bytes for the offset checker omitted from this lane. Tree sharing plus removal of redundant zero-offset fields reduces the monolithic data prototype from 456,548 to 220,643 bytes, or 51.67%. The split data/facade files actually shipped total 224,084 bytes, a 50.92% reduction after their additional composition overhead. These are representation measurements, not claimed reductions in the entire endpoint proof or its running time.

To fit the available memory, the finite well-formedness and certificate decisions are partitioned. All 70 selected project modules compiled during this session, with source, object, dependency and raw-log bindings retained. The packet contains a portable serial fresh-build runner. Historical failed large evaluations are marked separately and are not counted as successful checks. This partitioning is not a claim that the overall module count is smaller.

The composition itself illustrates another proof simplification: combine already checked Boolean facts through a generic lemma with abstract parameters. The helper `allC_of_halves` composes two child certificates without asking elaboration to normalize the entire concrete parent a second time. The semantic obligation is unchanged.

The final source-to-path equivalence reports only `propext` and `Quot.sound`. Composing with the existing path-to-iterative theorem additionally reports `Classical.choice`. Exact tree bindings and DAG well-formedness report only `propext`. No new axiom declaration, admitted theorem, or `native_decide` is introduced. The remaining production gate is to integrate the maintained generators and rebuild the affected binary/ZFC endpoint consumers; this source-level replay does not claim that larger rebuild.

9 What the new 214-state searches establish

We tested newly relevant final seeds, rather than rerunning only the inherited primary-K215 and append-one precursor searches. The main results are:

Seed	Inductive contexts	Specifications	Hits
CM215	972,655	315,436	0
K215-R41	980,047	316,292	0
Final ST215, append-zero	971,509	310,300	0

All three runs use radius 7 and fuel 97. The radius gives a 15-cell window. The domain starts with every window at initial control 0 and closes under both possibilities for newly encountered exterior bits. The closure is explicitly checked inductive. Candidate quotients merge a pair of states and try the specified inherited branch choices, including arbitrary remaining-target actions on a branch where the other action agrees.

A successful independently checked positive-block join would be an ordinary all-initial-tape simulation certificate: at a nonhalting join it would restore the full window, head, and mapped control before reading outside the known window. The checker also permits a terminal branch on which both runs halt; their final heads and tapes need not agree on that branch. This is stronger than bounded

blank-run agreement. But these runs found no successful candidate. A failed bounded join need not be a semantic counterexample; it can simply mean that the chosen relation, radius, or fuel is insufficient.

We also tested removal of CM215's selector 176 under the entire known source prefix at each of the eight relevant decoder occurrences. All 856 replacement actions targeting retained working states were tried. Direct transitions to halt are outside this enumeration and cannot satisfy its required nonhalting continuation contract. The bounds were 500 transitions after the first row, 24 additional cells, and 16,384 branch nodes per case. The existing selector is a positive control. No replacement passed.

A two-row fusion screen then considered every one of the 214 partner roles, with 860 possible actions per branch, including the terminal target. It performed 368,080 individual branch tests, allowing reentry into the already synthesized branch but rejecting a path that needs the unknown other branch before joining. This is an incomplete synthesis family, not an exhaustive enumeration of every two-row program. Only one branch survived, which is insufficient to assemble a merger.

Finally, a focused cutpoint search deleted 176 as an internal control and changed both 52/0 and 211/0. It did not insist that the removed internal control have a state-map image. Of 860 provider actions, five survived a preliminary screen, and all 4,300 completed entry/provider pairs failed the stated radius/fuel join test. Exact seeds, scripts, hashes, bounds, receipts, and reproduction commands are in `reduction_search/`.

10 A precise remaining selector problem

The most informative near miss concerns CM215's helper 176 and decoder role 211. Their read-one actions agree:

$$176/1 = 211/1 = (1, R, 129),$$

but their zero actions are

$$176/0 = (0, R, 185), \qquad 211/0 = (0, R, 159).$$

On every tape started at control 0, a direct incoming-edge argument establishes

$$q = 176 \wedge T(h) = 0 \Rightarrow T(h-1) = 1, \qquad q = 211 \Rightarrow T(h-1) = 0.$$

Thus the origin can again be distinguished by a predecessor bit when the current bit is zero. This is an ordinary structural fact, independently of the finite-window search.

For the first implication, 176 is entered only by 52/0, which moves left. If the bit then read at 176 is zero, the earlier entry into 52 must have been 165/0 or 193/0. The remaining entries write one, or arise from the exceptional path that forces the cell at 52 to be one and therefore cannot take 52/0. Every incoming edge to 165 and 193 writes one and moves right. Consequently the bit one more cell to the left is one. For the second implication, the only incoming edge to 211 is 220/0, which writes zero and moves right.

This supplies a second semantic marker-fold opportunity. It does not itself save a state. The simplest three-step selector needs unchanged providers with actions $0R159$ and $0R185$. In this exact table, the only providers are 211, which must be rewritten, and 176, which must be deleted. That is the specific obstruction to the simple template.

The focused search retained these five possible provider actions before completing the pair:

$$0L52, \quad 0L165, \quad 0L193, \quad 0R159, \quad 1L52.$$

None produced a passing completed pair in the stated family. A successful continuation may require a longer block, one more safe row rewrite, or a stronger source-level invariant. The failed tests do not exclude any of those possibilities.

This is a useful change in the next research objective: synthesize a certified two-context continuation for these exact endpoints, instead of searching blindly for duplicate rows. The desired local contract should be written first, including tape restoration, head displacement, and positive duration. If the synthesis must weaken the all-tape invariant to a source-reachable one, that weakening must be proved against the source program rather than inferred from a long blank trace.

11 A practical integration order

The following order keeps each review small and each failure attributable to one interface.

1. Add the reusable positive-simulation theorem, marker invariant, and exact dense transfer. Import the final CM215/ZFC adapter into the existing verified endpoint environment, compile it, and audit its assumptions. This is the shortest route to a second formal 215-state endpoint.
2. Add the Rocq mirror theorem and replace the computed backward certificate with its derived proof. Rebuild the affected source-equivalence and endpoint modules using the repository's pinned Rocq toolchain.
3. Adopt the path codec and its exact-certificate identity. Preserve input hashes and the old certificate as a temporary comparison fixture until the production rebuild passes.
4. Adopt shared tree literals. Regenerate the production file deterministically and rebuild all consumers whose source hashes changed. Keep the exact representation identities as small regression witnesses.
5. Adopt the ordinary zero-offset source checker only after its path decoder and finite-data binding pass. Judge success by source size, build cost, and a transparent semantic boundary, not solely by the number of generated modules.
6. Continue the 214-state work against the explicit selector contract above. A smaller source DAG or a passing finite prefix is a search lead, not a state-count or correctness result.

The first four changes are independent of a breakthrough in state count. They make subsequent candidate verification cheaper and easier to audit, which should improve the research loop even if the next machine still has 215 states.

12 Artifact and trust ledger

Artifact	Meaning and remaining boundary
`formal_cm215/`	New Lean positive simulation, local invariant, literal tables, Core embedding, and exact dense-machine equivalence. Contains unchanged minimal repository dependencies with pins and attribution. The final full-ZFC adapter is separate and was not compiled here.
`proof_audit/`	Exact source-sharing and path-codec generators, production-file prototypes, the shared-tree identity proof, measured sizes, and the audit of existing work. Production replacement remains subject to normal rebuild.
`coq/`	Exact original/compact certificate identity, actual-checker symmetry theorem, portable build helpers, and compiled/checker receipts. The local proof checker was Coq 8.18.0. Upstream K230 dependencies are resolved by exact paths and SHA-256 pins; no blanket license is assigned to them.

Artifact	Meaning and remaining boundary
`plain_source/`	The checked ordinary zero-offset source equivalence, retained logs, all 70 source-module bindings, and a portable fresh-build runner. Its status is explained in Section 8.
`reduction_search/`	New bounded negative searches, exact tables, a direct checker of the near-miss incidence facts, inherited search machinery, and reproduction commands. No 214-state machine or minimality theorem.
`evidence/`	Independent source-level reviews and machine/table comparisons. These are reviews within this collaboration, not claims of external human endorsement.
`report.tex`, `report.pdf`	This complete exposition, with separate build instructions and a hash manifest for the delivered files.

All new proof sources are supplied without `sorry`, `admit`, or a new axiom. The mathematical Lean transfer uses ordinary kernel decision procedures, not `native_decide`. The certificate generators, parsers, and searches are untrusted tools: exact kernel equalities or proved checker interfaces carry the formal claims. The existing logical calibration remains an explicit dependency, not a newly assumed consistency assertion.

Primary references and provenance

The repository source paths below are relative to the pinned ProveIt commit. They are sufficient to locate the exact definitions and records without relying on a moving branch.

[1] ProveIt snapshot: https://github.com/VladimirReshetnikov/ProveIt/tree/b6c7087477e51c017003bd729e712c8969733223.

[2] Existing K215 machine and ordinary proof: `Computability/BusyBeaver/Research/ZFC/candidate-k215/README.md` and `proof.md` in that directory.

[3] Latest simplification record: `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/README.md`, with the linked phase-three and original-build records.

[4] Exact Lean endpoint and current data: `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint215.lean`, `KSource215Data.lean`, `RMBisim.lean`, and `RMOffsetBisim.lean`.

[5] Exact Rocq source interface: `Computability/BusyBeaver/Research/ZFC/k225-formal/rocq/rocq-extension/coq/215/Source215.v`, `OffsetBisim230.v`, and `Cert215Offsets.v`.

[6] Existing CM215 construction: `Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/conditional-marker/README.md`, its tables, proof exposition, and retained attribution. The candidate predates this packet; the new contribution here is the compact compiled transfer proof and bridge.

[7] Other inspected final candidates: `candidate-k215/alternatives/rebalanced/README.md` and `candidate-k215/alternatives/direct-transfer/README.md`, relative to the ZFC research directory.

[8] Elliot Glazer's K230 verification, pinned at `3142e4dd980f0be12f4abb636c4ae2528afc17eb`: https://github.com/elliotglazer/k230-rocq/tree/3142e4dd980f0be12f4abb636c4ae2528afc17eb. The new Rocq symmetry proof imports its generic `RM.v` and `Bisim230.v`. Its retained `THIRD_PARTY.md` identifies the earlier Jeff/Mira-Cult and Wade dependencies and the applicable notice boundaries.